SQL JOINS

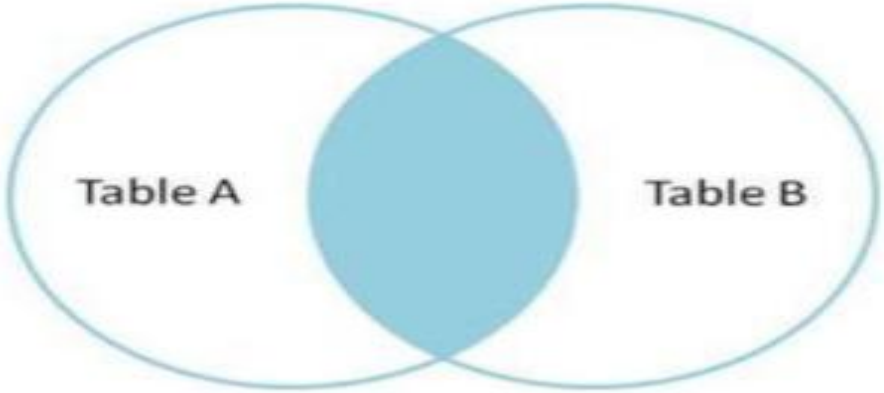
V P JAYACHITRA

SQL JOINS

- INNER vs OUTER
- Self Joins
- Cross Joins
- Join Rules

Inner Join

The most common type of join is SQL INNER JOIN (simple join).
An SQL INNER JOIN return all rows from multiple tables where the join condition is met.



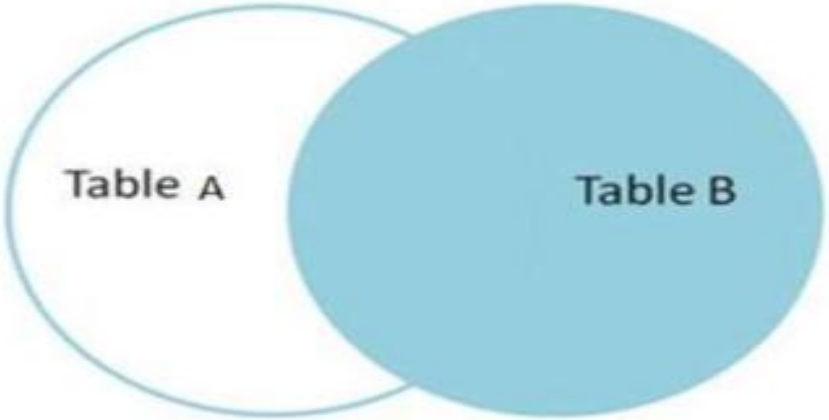
Syntax:

```
SELECT column_name(s)
FROM tableA
INNER JOIN tableB
ON tableA.column_name = tableB.column_name;
```

INNER JOIN is the same as JOIN.

Right outer join

The RIGHT JOIN keyword returns all rows from the right table (tableB), with the matching rows in the left table (tableA). The result is NULL in the left side when there is no match.



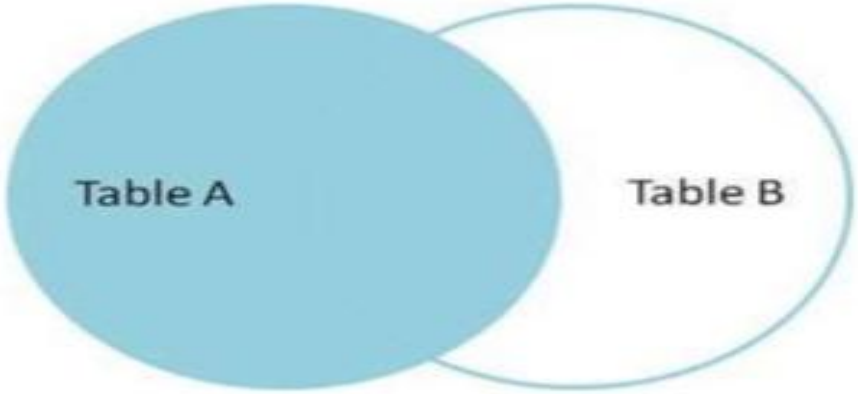
Syntax:

```
SELECT column_name(s)
FROM tableA
RIGHT JOIN tableB
ON tableA.column_name=tableB.column_name;
```

In some databases RIGHT JOIN is used only like RIGHT OUTER JOIN.

Left Outer Join

The LEFT JOIN keyword returns all rows from the left table (tableA), with the matching rows in the right table (tableB). The result is NULL in the right side when there is no match.



Syntax:

```
SELECT column_name(s)
FROM tableA
LEFT JOIN tableB
ON tableA.column_name = tableB.column_name;
```

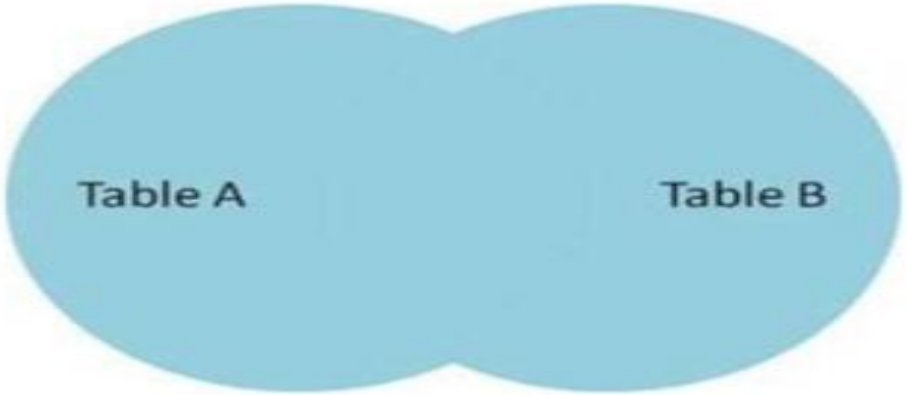
In some databases LEFT JOIN is used only like LEFT OUTER JOIN.

Full Outer Join

FULL OUTER JOIN

The FULL OUTER JOIN keyword returns all rows from the left table (tableA) and from the right table (tableB).

The FULL OUTER JOIN keyword combines the result of both LEFT and RIGHT joins.



Syntax:

```
SELECT column_name(s)
FROM tableA
FULL OUTER JOIN tableB
ON tableA.column_name=tableB.column_name;
```

CROSS JOIN

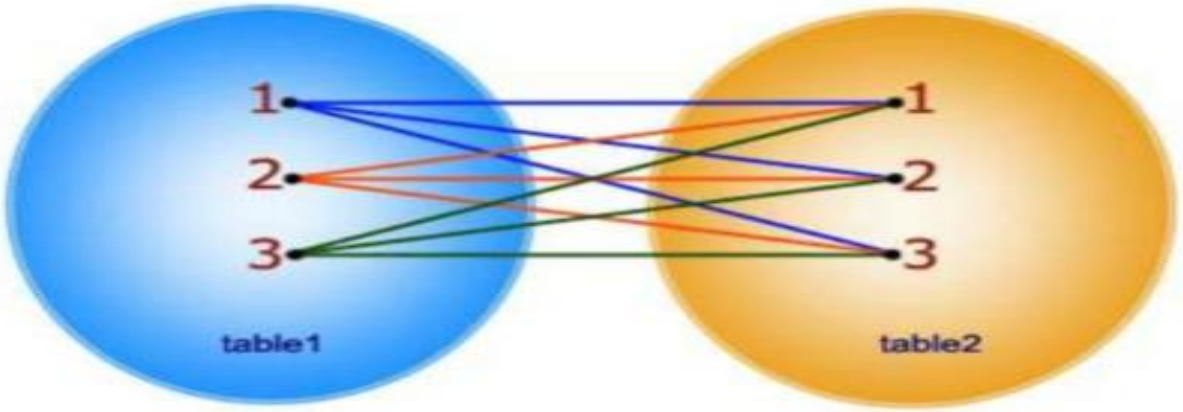
"All-to-All". The SQL CROSS JOIN produces a result set which is the number of rows in the first table multiplied by the number of rows in the second table. WHERE clause is not used along with CROSS JOIN. This kind of result is called as **Cartesian Product**.

```
SELECT *
FROM  tableA
CROSS JOIN tableB;
```

or

```
SELECT *
FROM tableA, tableB
```

SELECT * FROM table1 CROSS JOIN table2;



In CROSS JOIN, each row from 1st table joins with all the rows of another table. If 1st table contain x rows and y rows in 2nd one the result set will be x * y rows.

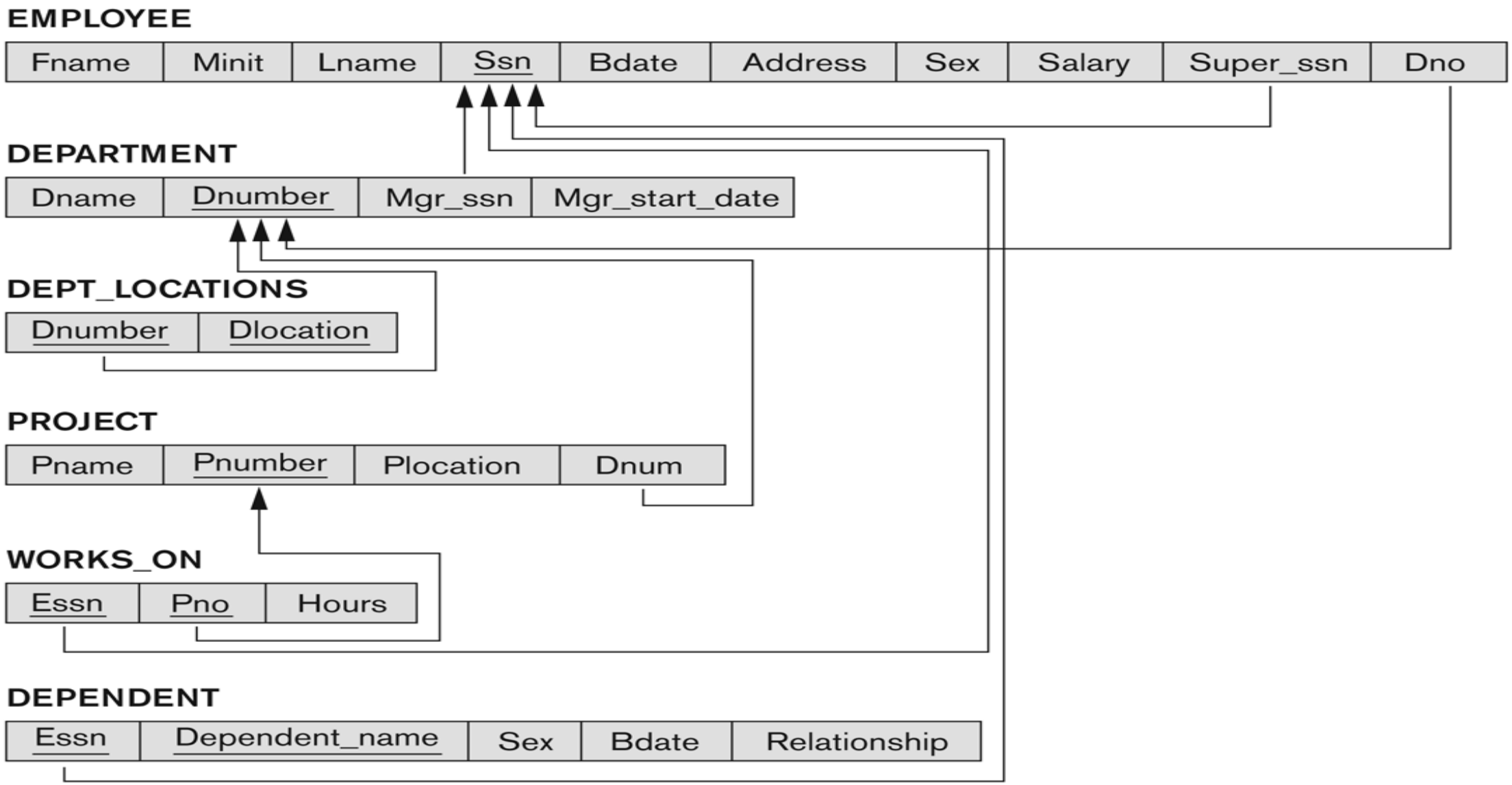
Join syntax

```
SELECT Attribute(s)
FROM TableA
{INNER | {LEFT | RIGHT | FULL} OUTER } JOIN
TableB
USING (join column list)
```

```
SELECT Attribute(s)
FROM TableA
{INNER | {LEFT | RIGHT | FULL} OUTER } JOIN
TableB
USING (join column list)
```

```
SELECT Attribute(s)
FROM TableA
NATURAL
{INNER | {LEFT | RIGHT | FULL}
OUTER } JOIN TableB
```

Figure 5.7
Referential integrity constraints displayed on the COMPANY relational database schema.



Why Joins AND Subqueries?

1. **Different Approaches, Same Result**
   Some queries work better with Joins, others with Subqueries

2. **Know When to Use Which**
   Joins: When you need columns from multiple tables

   Subqueries: When filtering based on calculated values

3. **Performance Matters**
   Understanding both helps you write efficient queries

What is a JOIN?

JOIN combines rows from two or more tables based on a related column (foreign key relationship).

EMPLOYEE Table:

```
Ssn | Fname | Dno
123 | John  | 5
456 | Alice | 4
```

DEPARTMENT Table:

```
Dnumber | Dname
5       | Research
4       | Admin
```

Types of JOINS

INNER JOIN	Returns ONLY matching rows from both tables
LEFT OUTER JOIN	All rows from LEFT table + matching from RIGHT
RIGHT OUTER JOIN	All rows from RIGHT table + matching from LEFT
FULL OUTER JOIN	All rows from BOTH tables (with NULLs)
SELF JOIN	Table joined with itself
CROSS JOIN	Cartesian product (all combinations)

JOIN Type	Question
INNER JOIN	"Show ONLY employees WHO HAVE valid departments"
LEFT OUTER JOIN	"Show ALL employees, including those WITHOUT departments"
RIGHT OUTER JOIN	"Show ALL departments, including those WITHOUT any employees"
FULL OUTER JOIN	"Show ALL employees AND ALL departments, whether matched or not"

INNER JOIN

```
SELECT  S.VENDORID, S.VENDORNAME, P.PONO, P.ITEM, P.QTY, P.PRICE
      FROM SUPPLIER89 S
      INNER JOIN PURCHASE89 P
      ON S.VENDORID = P.VENDORID;
```

Using Left Join

```
SELECT  S.VENDORID, S.VENDORNAME, P.PONO, P.ITEM, P.QTY, P.PRICE
      FROM SUPPLIER89 S
      LEFT JOIN PURCHASE89 P
      ON S.VENDORID = P.VENDORID;
```

RIGHT JOIN

```
SELECT  S.VENDORID, S.VENDORNAME, P.PONO, P.ITEM, P.QTY, P.PRICE
      FROM SUPPLIER89 S
      RIGHT JOIN PURCHASE89 P
      ON S.VENDORID = P.VENDORID;
```

```
SELECT  S.VENDORID, S.VENDORNAME, P.PONO, P.ITEM, P.QTY, P.PRICE
      FROM PURCHASE89 P
      RIGHT JOIN SUPPLIER89 S
      ON S.VENDORID = P.VENDORID;
```

FULL JOIN

```
SELECT  S.VENDORID, S.VENDORNAME, P.PONO, P.ITEM, P.QTY, P.PRICE
      FROM PURCHASE89 P
      FULL JOIN SUPPLIER89 S
      ON S.VENDORID = P.VENDORID;
```

INNER JOIN

Returns ONLY rows that have matching values in BOTH tables

Syntax:

```
SELECT E.Fname, D.Dname

FROM EMPLOYEE E

INNER JOIN DEPARTMENT D

ON E.Dno = D.Dnumber;
```

Note:
If an employee has Dno=NULL or a Dno that doesn't exist in DEPARTMENT,
that employee will NOT appear in the result!

LEFT OUTER JOIN (or LEFT JOIN)

Returns ALL rows from LEFT table + matching rows from RIGHT table (NULL if no match)

Syntax:

```
SELECT E.Fname, D.Dname

FROM EMPLOYEE E

LEFT JOIN DEPARTMENT D

ON E.Dno = D.Dnumber;
```

note:

When we need ALL employees, even those without a department.

Dname will be NULL for employees without matching department.

SELF JOIN

A table is joined with ITSELF using table aliases

Example: Find each employee with their supervisor's name

```
SELECT E.Fname AS Employee, S.Fname AS Supervisor
FROM EMPLOYEE E
LEFT JOIN EMPLOYEE S
ON E.Super_ssn = S.Ssn;
```

Use LEFT JOIN for self-joins when some employees might not have supervisors (CEO, etc.)

JOIN RULES

1. Always use table aliases (E, D, P) for readability

2. Join condition goes in ON clause, filtering goes in WHERE

3. Use INNER JOIN when you need matching rows only

4. Use LEFT/RIGHT JOIN when you need all rows from one table

5. Avoid CROSS JOIN unless you specifically need Cartesian product

Question 1: JOINS

Question:

Write a query to list all employees with their department names.
Include employees who don't have a department assigned.

Which JOIN should you use?

A) INNER JOIN

B) LEFT JOIN

C) RIGHT JOIN

D) CROSS JOIN

Question 1

Answer: B) LEFT JOIN

Solution:

```
SELECT E.Fname, E.Lname, D.Dname

FROM EMPLOYEE E

LEFT JOIN DEPARTMENT D

ON E.Dno = D.Dnumber;
```

Why LEFT JOIN?

- Returns ALL employees (left table)
- Includes matching departments where available
- Shows NULL for Dname if employee has no department

JOINs vs Subqueries

When to use JOIN vs Subquery?

Use JOIN when:

1. Need columns from multiple tables
2. Combining data is the main goal
3. Better performance (usually)
4. More readable for

Use SUBQUERY when:

1. Filtering based on calculated values
2. Need aggregation from another table
3. Checking existence (EXISTS)
4. Logical step-by-step

MCQ 1

Which JOIN returns all rows from the left table even if there's no match in the right table?

A) INNER JOIN

B) LEFT JOIN

C) RIGHT JOIN

D) CROSS JOIN

MCQ 1

Which JOIN returns all rows from the left table even if there's no match in the right table?

A) INNER JOIN

B) LEFT JOIN

C) RIGHT JOIN

D) CROSS JOIN

Answer: B) LEFT JOIN

LEFT JOIN returns ALL rows from the left table and matching rows from the right. Non-matching rows get NULL values.

MCQ 2

Which operator is used with multiple-row subqueries?

A) = (equals)

B) > (greater than)

C) IN

D) Both A and B

MCQ 2

Which operator is used with multiple-row subqueries?

A) = (equals)

B) > (greater than)

C) IN

D) Both A and B

Answer: C) IN

Multiple-row subqueries return multiple values. Use IN, ANY, or ALL operators. Single comparison operators (=, >, <) cause errors with multiple rows.

MCQ 3

Which statement about correlated subqueries is TRUE?

A) Inner query executes once

B) Inner query executes for each row of outer query

C) Cannot use EXISTS operator

D) Faster than non-correlated subqueries

MCQ 3

Which statement about correlated subqueries is TRUE?

A) Inner query executes once

B) Inner query executes for each row of outer query

C) Cannot use EXISTS operator

D) Faster than non-correlated subqueries

Answer: B) Inner query executes for each row of outer query

Correlated subqueries reference the outer query, so they must execute once for each row. EXISTS is commonly used with correlated subqueries.

MCQ 4

What does 'Salary > ALL (subquery)' mean?

A) Greater than at least one value

B) Greater than every value

C) Greater than the average

D) Equal to all values

MCQ 4

What does 'Salary > ALL (subquery)' mean?

A) Greater than at least one value

B) Greater than every value

C) Greater than the average

D) Equal to all values

Answer: B) Greater than every value

ALL means the condition must be true for ALL values. > ALL means greater than the maximum value. > ANY means greater than the minimum.

MCQ 5

When would you use EXISTS instead of IN?

A) When checking for exact value matches

B) When checking if related rows exist

C) When subquery returns single value

D) When using aggregate functions

MCQ 5

When would you use EXISTS instead of IN?

A) When checking for exact value matches

B) When checking if related rows exist

C) When subquery returns single value

D) When using aggregate functions

Answer: B) When checking if related rows exist

EXISTS is used with correlated subqueries to check if ANY rows exist that match a condition. It's more efficient than IN for large datasets.

SQL-set operations

- In SQL, there are three operations
  - UNION
  - INTERSECT
  - EXCEPT
- In addition, there are multiset operations, that do not eliminate duplicates
  - UNION ALL
  - INTERSECT ALL
  - EXCEPT ALL

SET OPERATIONS

- **SQL has directly incorporated some set operations**
- **There is a union operation (UNION), and in *some versions* of SQL there are set difference (MINUS) and intersection (INTERSECT) operations**
- **The resulting relations of these set operations are sets of tuples; *duplicate tuples are eliminated from the result***
- **The set operations apply only to *union compatible relations*;**
  - **The two relations must have the same attributes and the attributes must appear in the same order**

SET OPERATIONS (contd.)

- **Query 4: Make a list of all project numbers for projects that involve an employee whose last name is 'Smith' as a worker or as a manager of the department that controls the project.**

Q4:

```
(SELECT  PNAME
FROM     PROJECT, DEPARTMENT, EMPLOYEE
WHERE    DNUM=DNUMBER AND MGRSSN=SSN AND LNAME='Smith')
UNION
(SELECT  PNAME
FROM     PROJECT, WORKS_ON, EMPLOYEE
WHERE    PNUMBER=PNO AND ESSN=SSN AND NAME='Smith‘)
```

NESTING OF QUERIES

- **A complete SELECT query, called a *nested query*, can be specified within the WHERE-clause of another query, called the *outer query***
  - **Many of the previous queries can be specified in an alternative form using nesting**
- **Query 1: Retrieve the name and address of all employees who work for the 'Research' department.**

```
Q1: SELECT  FNAME, LNAME, ADDRESS
    FROM    EMPLOYEE
    WHERE DNO IN  (SELECT  DNUMBER
                   FROM DEPARTMENT
                   WHERE DNAME='Research' )
```

NESTING OF QUERIES (contd.)

- **The nested query selects the number of the 'Research' department**
- **The outer query select an EMPLOYEE tuple if its DNO value is in the result of either nested query**
- **The comparison operator IN compares a value v with a set (or multi-set) of values V, and evaluates to TRUE if v is one of the elements in V**
- **In general, we can have several levels of nested queries**
- **A reference to an *unqualified attribute* refers to the relation declared in the *innermost nested query***
- **In this example, the nested query is *not correlated* with the outer query**

Retrieve the name of each employee who has a dependent with the same first name as the employee.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

CORRELATED NESTED QUERIES

- **If a condition in the WHERE-clause of a *nested query* references an attribute of a relation declared in the *outer query*, the two queries are said to be *correlated***
  - **The result of a correlated nested query is different for each tuple (or combination of tuples) of the relation(s) the outer query**
- **Query 12: Retrieve the name of each employee who has a dependent with the same first name as the employee.**

```
Q12: SELECT  E.FNAME,  E.LNAME
     FROM EMPLOYEE AS E
     WHERE  E.SSN IN
                (SELECT   ESSN
                 FROM     DEPENDENT
                 WHERE    ESSN=E.SSN AND  E.FNAME=DEPENDENT_NAME)
```

```
SELECT  E.FNAME, E.LNAME
FROM   EMPLOYEE E, DEPENDENT D
WHERE  E.SSN=D.ESSN AND
       E.FNAME=D.DEPENDENT_NAME
```

THE EXISTS FUNCTION (contd.)

- **EXISTS is used to check whether the result of a correlated nested query is empty (contains no tuples) or not**
- Query 12: **Retrieve the name of each employee who has a dependent with the same first name as the employee.**
  Query 12 in an alternative form that uses EXISTS as Q12B

```
Q12B:   SELECT      FNAME, LNAME
        FROM        EMPLOYEE
        WHERE EXISTS (SELECT *
                      FROM   DEPENDENT
                      WHERE SSN=ESSN AND FNAME=DEPENDENT_NAME)
```

```
Q12: SELECT  E.FNAME, E.LNAME
     FROM EMPLOYEE AS E
     WHERE E.SSN IN
         (SELECT  ESSN
         FROM DEPENDENT
         WHERE  ESSN=E.SSN AND E.FNAME=DEPENDENT_NAME)
```

THE EXISTS FUNCTION (contd.)

- Query 6: **Retrieve the names of employees who have no dependents.**

```
Q6:     SELECT        FNAME, LNAME
        FROM          EMPLOYEE
        WHERE NOT EXISTS  (SELECT   *
                           FROM   DEPENDENT
                           WHERE        SSN=ESSN)
```

- **In Q6, the correlated nested query retrieves all DEPENDENT tuples related to an EMPLOYEE tuple. If *none exist*, the EMPLOYEE tuple is selected**
  - **EXISTS is necessary for the expressive power of SQL**

CORRELATED NESTED QUERIES (contd.)

- **The CONTAINS comparison operator, which is used in conjunction with nested correlated queries**
  - **This operator was *dropped from the language*, possibly because of the difficulty in implementing it efficiently**
  - **Most implementations of SQL do not have this operator**
  - **The CONTAINS operator compares *two sets of values*, and returns TRUE if one set contains all values in the other set**
    - **Reminiscent of the division operation of algebra**

CORRELATED NESTED QUERIES (contd.)

- **Query 3: Retrieve the name of each employee who works on all the projects controlled by department number 5.**

Q3:

```
SELECT    FNAME, LNAME
FROM      EMPLOYEE
WHERE (   (SELECT    PNO
          FROM       WORKS_ON
          WHERE      SSN=ESSN)
               CONTAINS
          (SELECT    PNUMBER
          FROM       PROJECT
          WHERE      DNUM=5) )
```

In Q3, the second nested query, which is *not correlated* with the outer query, retrieves the project numbers of all projects controlled by department 5

The first nested query, which is correlated, retrieves the project numbers on which the employee works, which is *different for each employee tuple* because of the correlation

Joined Relations

- **Can specify a "joined relation" in the FROM-clause**
  - **Looks like any other relation but is the result of a join**
  - **Allows the user to specify different types of joins (regular "theta" JOIN, NATURAL JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, CROSS JOIN, etc)**

Retrieve the employees and their managers

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Joined Relations

- **Examples:**

```
Q8:SELECT    E.FNAME, E.LNAME, S.FNAME, S.LNAME
   FROM      EMPLOYEE E S
   WHERE     E.SUPERSSN=S.SSN
```

- **can be written as:**

```
Q8:SELECT    E.FNAME, E.LNAME, S.FNAME, S.LNAME
   FROM      (EMPLOYEE E LEFT OUTER JOIN EMPLOYEES
   ON        E.SUPERSSN=S.SSN)
```

Joined Relations

- **Examples:**

```
Q1:SELECT   FNAME, LNAME, ADDRESS
   FROM     EMPLOYEE, DEPARTMENT
   WHERE          DNAME='Research' AND DNUMBER=DNO
```

- **could be written as:**

```
Q1:SELECT          FNAME, LNAME, ADDRESS
   FROM            (EMPLOYEE JOIN DEPARTMENT
                   ON DNUMBER=DNO)
   WHERE           DNAME='Research’
```

- **or as:**

```
Q1:SELECT     FNAME, LNAME, ADDRESS
   FROM            (EMPLOYEE NATURAL JOIN DEPARTMENT
                   AS DEPT(DNAME, DNO, MSSN, MSDATE)
   WHERE           DNAME='Research’
```

Retrieve the employees detail of research department

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

```
SELECT    FNAME, LNAME, ADDRESS
FROM      (EMPLOYEE NATURAL JOIN DEPARTMENT
          AS DEPT(DNAME, DNO, MSSN, MSDATE)
          WHERE    DNAME='Research'
```

Joined Relations

- **illustrates multiple joins in the joined tables**

Q2:
```
SELECT PNUMBER, DNUM, LNAME,BDATE, ADDRESS
FROM  ((PROJECT
JOIN    DEPARTMENT
ON      DNUM=DNUMBER)
JOIN    EMPLOYEE
ON      MGRSSN=SSN)
WHERE PLOCATION='Stafford’
```